FULL STACK DEVELOPMENT / SERVICENOW INTEGRATION PROJECT DOCUMENTATION

Team ID: SWTID-2026-1398	**Date:** 30 September 2026
Project Title: Import Data Using Transform Maps	**Domain:** ServiceNow Integration & Automation

1. Introduction

- • **Project Title:** Import Data Using Transform Maps
- • **Team Members & Roles:**
  Full Stack & ServiceNow Integration Development Team
- **- Lead Developer (ID:** SWTID-2026-1398 — Project Lead & Integration Architect (ServiceNow Administration, Import Sets, Transform Map Design, Coalesce Logic Setup)
- **- Backend & Data Specialist (ID:** SWTID-2026-1398 — Full Stack Engineer (MERN Interface, Staging API Endpoints, Automated File Parsing & Validation Engine)

2. Project Overview

2.1 Purpose

The primary objective of this project is to automate and streamline employee data ingestion into ServiceNow using Import Sets and Transform Maps. Spreadsheet data containing employee records (including Employee ID, Name, Email, Department, and Location) is uploaded and staged within a temporary Import Set Table (u_imp_employee). A configured Transform Map maps staging fields directly into the target Employee table (sys_user). Using Employee ID as a unique Coalesce identifier, the system automatically detects existing employee records and updates them, while seamlessly inserting new employee entries without record duplication.

2.2 Key Features & Functionalities

- • **Automated File Upload & Staging:** Supports multi-format spreadsheet ingestion (.xlsx, .csv) with schema validation prior to processing.
- • **Import Set Staging:** Temporary staging storage in an Import Set Table ensuring raw data is isolated and validated prior to database commit.
- • **Dynamic Field Mapping:** Flexible field mapping between source spreadsheet columns and target sys_user table attributes.

- • **Smart Record Upsert (Coalesce Logic):** Employee ID serves as a Coalesce key to run upsert operations (updating existing employee profiles and inserting new records).
- • **Audit Logging & Reporting:** Detailed log generation tracking import executions, transformation successes, and error records for audit purposes.

3. Architecture

3.1 Frontend Architecture (React / Client)

The frontend is constructed using React.js with a component-driven architecture. It provides an intuitive user interface for administrators to upload employee spreadsheets, monitor real-time transformation progress, review field mapping schemas, and inspect batch import logs. Axios is used for asynchronous API communication with the backend staging service.

3.2 Backend Architecture (Node.js & Express.js)

The backend service is powered by Node.js and Express.js. It acts as an intermediate middleware layer responsible for accepting spreadsheet uploads via Multer, parsing file contents, executing initial data validation routines, and communicating with the ServiceNow REST Table API / Import Set API to push records into the u_imp_employee staging table.

3.3 Database & ServiceNow Target Schema

The database setup operates across two main tiers:
1. MongoDB (Local/App Database): Stores application metadata, user authentication tokens, audit trail histories, and cached import job statuses.
2. ServiceNow Instance Tables:
  - Import Set Table (u_imp_employee): Staging table holding raw incoming rows.
  - Target Employee Table (sys_user): System user database where transformation results are committed.

4. Setup Instructions

4.1 Prerequisites

- • **Node.js:** v18.x or higher
- • **MongoDB:** v6.0 or higher (Local or MongoDB Atlas)
- • **ServiceNow Instance:** Personal Developer Instance (PDI) with admin privileges
- • **npm / yarn:** v10.x or higher

4.2 Installation & Setup Guide

Step 1: Clone the Repository & Install Dependencies

```
git clone https://github.com/SWTID-2026-1398 /import-data-transform-maps.git
cd import-data-transform-maps
```

```
# Install Backend Dependencies
cd server && npm install

# Install Frontend Dependencies
cd ../client && npm install
```

Step 2: Configure Environment Variables (.env)

```
# server/.env
PORT=5000
MONGO_URI=mongodb://localhost:27017/servicenow_import_db
JWT_SECRET=super_secret_jwt_key_2026
SN_INSTANCE_URL=https://dev5778.service-now.com
SN_USER=admin
SN_PASSWORD=your_instance_password
```

5. Folder Structure

5.1 Project Layout

```
import-data-transform-maps/
├── client/                        # React Frontend
│   ├── src/
│   │   ├── components/          # Navigation, Header, Sidebar
│   │   ├── pages/               # Dashboard, UploadPage, LogsPage
│   │   ├── services/            # Axios API calls
│   │   ├── App.js               # Main Routing
│   │   └── index.js             # React Entry Point
│   └── package.json
└── server/                        # Node.js / Express Backend
    ├── config/                  # DB and ServiceNow connection configs
    ├── controllers/             # Import & Transform Map controllers
    ├── middleware/              # Multer upload & Auth middleware
    ├── models/                  # MongoDB Schemas (ImportLog, User)
    ├── routes/                  # Express REST API routes
    ├── utils/                   # Excel parser & SN API helper
    ├── server.js                # Express App Server Entry
    └── package.json
```

6. Running the Application

Execute the following commands in separate terminal instances to launch the local development environment:

6.1 Starting the Backend Server

```
cd server
npm start
# Output: Server running on port 5000 | Connected to MongoDB
```

6.2 Starting the Frontend Client

```
cd client
npm start
# Output: Compiled successfully! Local: http://localhost:3000
```

7. API Documentation

The backend exposes RESTful API endpoints for spreadsheet ingestion and ServiceNow synchronization:

Method	Endpoint	Auth Required	Description
POST	/api/upload/excel	Yes (JWT)	Uploads .xlsx file, parses rows, and pushes to ServiceNow staging table.
POST	/api/transform/execute	Yes (JWT)	Triggers Transform Map processing in ServiceNow using Coalesce key.
GET	/api/logs/history	Yes (JWT)	Fetches batch import log records and transformation status.
POST	/api/auth/login	No	Authenticates admin users and returns JWT access token.

8. Authentication & Authorization

Authentication is enforced using JSON Web Tokens (JWT) for the React-Node application layer and HTTP Basic Auth / OAuth 2.0 for ServiceNow REST communications:

• User Auth: System administrators log in via the web application to receive a signed JWT token valid for 8 hours.

• Protected API Routes: All import and transform trigger endpoints require a valid Bearer Token in the HTTP Authorization header.

• ServiceNow Auth: Middleware securely encrypts and manages instance service credentials to authenticate against ServiceNow REST Table APIs.

9. User Interface

The web application features a clean UI built with React and Tailwind CSS:
1. Upload Portal: Drag-and-drop spreadsheet upload zone with instant pre-validation error checking.
2. Transform Map Configuration View: Displays target field alignments (Employee ID -> sys_user.employee_number, Name -> sys_user.name, Email -> sys_user.email, Department -> sys_user.department, Location -> sys_user.location).
3. Import Execution & Status Dashboard: Real-time progress bar showing insert/update operation counters.

10. Testing Strategy

A multi-tier testing methodology was executed to ensure end-to-end data integrity:

- • **Unit Testing:** Tested Express routes and Excel parsing utility functions using Jest and Supertest.
- • **Integration Testing:** Verified API communication between Express server and ServiceNow Import Set endpoint.
- • **Transformation Verification Testing:** Ingested sample spreadsheets containing 500+ employee records with deliberate duplicates to confirm that existing records were updated and new records were inserted accurately based on the Employee ID Coalesce rule.

11. Screenshots or Demo

- • **Project Repository:** https://github.com/SWTID-2026-1398 /import-data-transform-maps
- • **Live ServiceNow Demo Instance:** https://dev5778.service-now.com (ServiceNow Instance Import Set & Transform Map Workbench)
- • **Screen Captures:** Included in Appendix section and repository README.md.

12. Known Issues

- • **Large File Processing Delay:** Spreadsheets containing over 10,000 rows may cause high memory consumption during synchronous parsing; recommended batching size is 2,000 rows per payload.
- • **Unmatched Reference Fields:** If a department string in the spreadsheet does not exist in the target ServiceNow instance, reference field creation depends on Transform Map choice scripts.

13. Future Enhancements

- • **Scheduled Import Jobs:** Implement automated background scheduling using BullMQ / Redis queues for periodic spreadsheet polling.
- • **Interactive Mapping Canvas:** Add dynamic visual canvas allowing administrators to draw custom field mappings without modifying transform scripts.

- • **Automated Notification Center:** Send instant Email / Slack alerts to system admins upon transform map completion or error detection.